Superpowers + Everything Claude Code SDLC Workflow for Feature Development

This document describes a **full SDLC workflow for feature development** using:

- the **Superpowers** plugin for Claude Code (spec- and plan-first discipline), and
- **Everything Claude Code (ECC)** as the configurable agent + skills framework for architecture, planning, development, review, security, and testing.

The workflow is requirements-driven:

1. Software requirement analysis (BRS → SRS → SRS review).
2. Architecture (with a dedicated architecture agent and security/edge-case review).
3. Planning (with `simplify` and `ddd_verification`).
4. Development (again with `simplify` and `ddd_verification`, plus TDD).
5. Testing (BRS/SRS-based test design, unit + integration + endpoint + Playwright E2E).

You can adapt this as a **.md reference in your repo** and as a **team SOP** for how to use Claude Code when adding or changing features.

---

1. Building blocks: what Superpowers and ECC provide

1.1 Superpowers (Claude Code plugin)

Superpowers is an official Claude Code plugin that enforces a **structured 4-phase workflow** for coding tasks:

1. **Brainstorm** – clarify requirements, constraints, and design choices before writing code.
2. **Git worktree** – work on an isolated worktree/branch for the change.
3. **Write plan** – break the work into ordered, test-first tasks.
4. **Execute plan** – run those tasks via sub-agents using TDD and code review.

For this doc, we primarily use two commands:

- `/brainstorming` – spec & design clarifier.
- `/execute-plan` – executes a pre-approved plan, ensuring each step has a failing test, a fix, and verification.

Internally, Superpowers provides skills for TDD, debugging, subagent development, and spec- and plan-quality enforcement. You don't call these skills directly; you trigger them through the commands above.

1.2 Everything Claude Code (ECC)

ECC is a **Claude Code configuration framework** that provides:

- **Agents** – e.g. `planner`, `architect`, `tdd-guide`, `code-reviewer`, `security-reviewer`, `e2e-runner`, `doc-updater`, `build-error-resolver`.
- **Skills** – e.g. `tdd-workflow`, `backend-patterns`, `frontend-patterns`, `security-review`, `e2e-testing`, `deployment-patterns`, `database-migrations`, `search-first`.
- **Commands** – e.g. `/plan`, `/tdd`, `/code-review`, `/security-scan`, `/e2e`, `/test-coverage`, `/verify`, `/update-docs`, `/orchestrate`.

ECC is intended as a **general SDLC harness**. You extend it with your own domain-specific skills (BRS, SRS, DDD verification, simplify, test design, etc.), and then compose workflows using its commands.

---

2. Custom skills you should add on top of ECC

For the SDLC you're building, you should define the following **custom skills** in addition to ECC's built-in skills. Conceptually, each custom skill is a "toolbox + prompt contract" that ECC agents can call.

2.1 Requirements skills

- brs-generation
  - Input: stakeholder notes, user stories, existing docs.
  - Output: docs/brs.md (business requirements), including business goals, actors, scope, constraints, KPIs.
- srs-generation
  - Input: docs/brs.md, repo context, domain glossary.
  - Output: docs/srs.md (software requirements), with FRs, NFRs, acceptance criteria, IDs.
- requirements-review
  - Input: docs/srs.md, optionally docs/brs.md.
  - Output: docs/requirements-review.md listing ambiguities, missing NFRs, conflicts, and suggested fixes.
- traceability-matrix
  - Input: docs/brs.md, docs/srs.md.
  - Output: docs/requirements-traceability.csv mapping BR → SR → test IDs.

2.2 Architecture and DDD skills

- edge-case-review
  - Input: docs/architecture.md, docs/srs.md.
  - Output: docs/edge-cases.md listing failure modes, error paths, abuse cases, race conditions, and mitigation ideas.
- simplify
  - Input: docs/plan.md or code diffs.
  - Output: docs/simplify-review.md or inline suggestions; flags overengineering, unnecessary patterns, and complexity.
- ddd-verification
  - Input: docs/architecture.md, docs/plan.md, docs/srs.md, or code.

    - Output: docs/ddd-verification.md; checks bounded contexts, aggregate boundaries, ubiquitous language, invariants, event semantics.

2.3 Testing skills

- test-design
    - Input: docs/brs.md, docs/srs.md.
    - Output: docs/test-cases.md listing unit/integration/endpoint/e2e test cases linked to requirement IDs.
- endpoint-tests
    - Input: API contract and docs/test-cases.md.
    - Output: concrete API test code in your language/framework.

You can implement these as separate skill prompt files in ECC's skills/ directory and then wire them into commands or agents as needed.

---

3. Stage-by-stage workflow with commands and skills

3.1 Software requirement analysis (BRS → SRS)

Goal: Turn stakeholder intent into testable, traceable software requirements before any architecture or implementation.

Step 3.1.1 – High-level requirement and context clarification (Superpowers)

Command:

```
/brainstorming
```

Use this when:

- A new feature or epic is proposed.
- Requirements are fuzzy or underspecified.

What you do:

- Explain the business problem, users, constraints, and rough idea of the feature.
- Let Superpowers drive a structured brainstorming and converge on a written spec.

What you get:

- A Superpowers-generated spec that clarifies flows, edge cases, and constraints.

Step 3.1.2 – Generate BRS (ECC + custom skill)

Command (conceptual):

```
/orchestrate brs-generation
```

Underlying skills/agents:

- `brs-generation` skill
- `search-first` skill (to pull from repo/docs if needed)

Inputs:

- Output of `/brainstorming`.
- Stakeholder notes, existing docs.

Outputs:

- `docs/brs.md` (Business Requirements Specification).

Step 3.1.3 – Generate SRS (ECC + Problem-Based-SRS + custom skill)

Command:

```
/orchestrate srs-generation using docs/brs.md
```

Underlying skills/agents:

- `srs-generation` (which internally calls Problem-Based-SRS if you integrate it).

Inputs:

- docs/brs.md
- Repo/application context

Outputs:

- docs/srs.md (Software Requirements Specification).

Step 3.1.4 – Review and traceability (ECC)

Commands:

```
/orchestrate requirements-review on docs/srs.md
/orchestrate traceability-matrix build docs/brs.md docs/srs.md
```

Outputs:

- docs/requirements-review.md
- docs/requirements-traceability.csv

At the end of this stage, you must approve:

- brs.md
- srs.md
- requirements-traceability.csv

before letting any architecture or planning work start.

3.2 Architecture stage

Goal: Design or adjust the architecture so it satisfies the SRS, aligns with BRS intent, and is secure and testable.

Step 3.2.1 – Invoke architecture agent (ECC)

Command:

```
/plan "Design or update architecture for <feature> based on docs/brs.md and
```

What happens:

- ECC uses the planner and architect agents:
  - Reads docs/brs.md, docs/srs.md, repo context.
  - Produces docs/architecture.md and
  - Potential ADRs in docs/adr/.

Step 3.2.2 – Architecture review and edge cases (ECC + custom skills)

Commands:

```
/orchestrate architecture-review on docs/architecture.md
/orchestrate edge-case-review on docs/architecture.md docs/srs.md
/orchestrate security-review on docs/architecture.md
```

Skills/agents involved:

- ECC architect + code-reviewer or a dedicated architecture-reviewer agent.
- edge-case-review custom skill.
- security-review skill and security-reviewer agent.

Outputs:

- docs/architecture-review.md
- docs/edge-cases.md
- docs/security-review.md

Only when architecture, security, and edge-case reviews are acceptable should planning start.

3.3 Planning stage (simplify + ddd_verification)

Goal: Turn architecture and SRS into an implementation plan with minimal complexity and sound DDD boundaries.

Step 3.3.1 – Create an implementation plan (ECC)

Command:

```
/plan "Implement <feature> according to docs/architecture.md and docs/srs.r
```

Outputs:

- docs/plan.md – epics, stories, tasks, and milestones.

Step 3.3.2 – Run simplify and DDD checks (ECC + custom skills)

Commands:

```
/orchestrate simplify on docs/plan.md
/orchestrate ddd-verification on docs/plan.md docs/architecture.md docs/srs.
```

Outputs:

- docs/simplify-review.md – flags overengineering, suggests smaller slices.
- docs/ddd-verification.md – checks that tasks align with bounded contexts and domain rules.

If simplify/DDD checks find issues, adjust docs/plan.md and rerun until clean.

3.4 Development stage (simplify + ddd_verification + TDD)

Goal: Implement the feature through small TDD cycles, keeping code aligned with design and domain.

For each story or task from docs/plan.md:

Step 3.4.1 – Use Superpowers if you want local spec/plan per story (optional)

For large or tricky stories, you can:

```
/brainstorming
```

- Give Superpowers the story description.
- Let it generate a mini-spec and plan.

Then either:

- Use `/execute-plan` directly for that local chunk, or
- Transcribe that plan into ECC `/plan` flows.

Step 3.4.2 – TDD implementation (ECC)

Command:

```
/tdd
```

What happens:

- `tdd-guide` agent and `tdd-workflow` skill orchestrate:
  - writing failing unit/integration tests tied to SRS IDs,
  - implementing minimal code to make them pass,
  - refactoring while preserving behavior.

You should feed in:

- The relevant SRS sections (SR-XXX, NFR-YYY).
- Relevant architecture notes.

Step 3.4.3 – Simplify + DDD verification on code (ECC + custom skills)

After a code chunk is done:

Commands:

```
/orchestrate simplify on <changed files>
/orchestrate ddd-verification on <changed files> docs/architecture.md docs/sr
```

Outputs:

- Suggestions to remove unneeded abstractions.
- Warnings if code violates domain boundaries or uses wrong terminology.

Step 3.4.4 – Code review and security review (ECC)

Commands:

```
/code-review
/security-scan
```

- `/code-review` uses `code-reviewer` agent to look for maintainability and correctness problems.
- `/security-scan` uses `security-reviewer` + `security-review`/AgentShield skills for vulnerabilities and misconfigurations.

Only when:

- TDD cycles are green,
- simplify/DDD checks look good,
- code and security reviews pass,

you move on to full test runs and integration.

3.5 Testing stage (BRS/SRS-driven + unit/integration/endpoint/E2E)

Goal: Prove that implementation satisfies business and software requirements via layered tests.

Step 3.5.1 – Derive test cases from BRS and SRS (ECC + custom skill)

Command:

```
/orchestrate test-design using docs/brs.md docs/srs.md
```

Outputs:

- `docs/test-cases.md` – each test case linked to BR and SR IDs, plus test level (unit/integration/endpoint/e2e).

Step 3.5.2 – Implement unit and integration tests (ECC)

This is partially handled during `/tdd`, but you can also run explicit commands:

- For backend/domain logic:
  - Use `/tdd` again focused on specific modules.
- For integration (DB, third-party APIs, queues):
  - Use `/tdd` with integration-focused prompts.

Step 3.5.3 – Implement endpoint/API tests (ECC + custom `endpoint-tests`)

Command:

```
/orchestrate endpoint-tests using docs/test-cases.md
```

Outputs:

- Endpoint/API tests aligned to SRS and contract.

Step 3.5.4 – Frontend Playwright tests (ECC `e2e-runner`)

Command:

```
/e2e
```

What it does:

- Uses `e2e-runner` agent and `e2e-testing` skill to:
  - create or update Playwright tests,
  - cover key user flows (e.g., invoice creation, validation errors, success states),
  - incorporate BRS/SRS scenarios.

Step 3.5.5 – Run coverage and verification gates (ECC)

Commands:

```
/test-coverage
```

```
/verify
```

- /test-coverage analyzes coverage and flags missing tests.
- /verify runs the configured verification loop to ensure all checks (tests, review, security, requirements) are satisfied before merge.

---

4. End-to-end example: implementing an "Invoice Draft" feature

4.1 Context

You want to add "Invoice Draft" capability to an existing invoicing system.

4.2 Command sequence overview

```
# Requirements
/brainstorming                    # Superpowers: clarify invoice draft feature
/orchestrate brs-generation             # ECC: produce docs/brs.md
/orchestrate srs-generation using docs/brs.md
/orchestrate requirements-review on docs/srs.md
/orchestrate traceability-matrix build docs/brs.md docs/srs.md

# Architecture
/plan "Design or update architecture for Invoice Draft based on docs/brs.md
/orchestrate architecture-review on docs/architecture.md
/orchestrate edge-case-review on docs/architecture.md docs/srs.md
/orchestrate security-review on docs/architecture.md

# Planning
/plan "Implement Invoice Draft feature according to docs/architecture.md an
/orchestrate simplify on docs/plan.md
/orchestrate ddd-verification on docs/plan.md docs/architecture.md docs/srs.

# For each story in docs/plan.md
/tdd                              # ECC TDD cycles for that story
/orchestrate simplify on <changed files>
```

```
/orchestrate ddd-verification on <changed files> docs/architecture.md docs/sr
/code-review
/security-scan

# Testing
/orchestrate test-design using docs/brs.md docs/srs.md
/orchestrate endpoint-tests using docs/test-cases.md
/e2e                          # Playwright tests for invoice draft UI flow
/test-coverage
/verify

# Documentation
/update-docs
```

4.3 What each stage produces for this feature

- `docs/brs.md` – Business goals for invoice drafts (who, why, KPIs).
- `docs/srs.md` – Exact behavior: create draft, edit, auto-save, validation rules, security.
- `docs/requirements-traceability.csv` – mapping BR ↔ SR ↔ test.
- `docs/architecture.md` – changes to services, DB schema, endpoints, events.
- `docs/edge-cases.md` – concurrent edits, partial failures, invalid tax profiles.
- `docs/security-review.md` – data exposure, roles, permissions, rate limiting.
- `docs/plan.md` – epics/stories/tasks for invoice draft feature.
- `docs/ddd-verification.md` – domain checks (Invoice aggregate, Customer, Tax).
- `docs/test-cases.md` – specific tests per SR.
- Code + tests + Playwright specs.
- `docs/test-report.md` + coverage and verification logs.

---

5. Quick cheat sheet

5.1 Which commands to run in which stage

Stage	Main commands	Role
Requirement analysis	/brainstorming, /orchestrate brs-generation, /orchestrate srs-generation, /orchestrate requirements-review, /orchestrate traceability-matrix	Clarify, formalize, and review requirements before design.
Architecture	/plan "Design/update architecture...", /orchestrate architecture-review, /orchestrate edge-case-review, /orchestrate security-review	Design system, review risks, document edge cases and security.
Planning	/plan "Implement <feature>...", /orchestrate simplify, /orchestrate ddd-verification	Break feature into DDD-aligned tasks, reduce complexity.
Development	/tdd, /orchestrate simplify, /orchestrate ddd-verification, /code-review, /security-scan	Implement via TDD, keep design simple and domain-correct, review and secure.
Testing	/orchestrate test-design, /orchestrate endpoint-tests, /e2e, /test-coverage, /verify	Derive tests from requirements, cover endpoints and UI, enforce gates.
Documentation	/update-docs	Keep documentation aligned with new behavior.

5.2 Mental model

- **Superpowers** owns the **top-level thinking** – it forces you to stop and design/spec before code.
- **ECC** owns the **SDLC plumbing** – agents, skills, and commands for architecture, TDD, review, security, and testing.
- **Your custom skills** (BRS, SRS, DDD, simplify, test-design) plug into ECC so that every Claude Code interaction respects the SDLC you want, not just ad-hoc coding.